

CO2 AT1

NAME: I Raiyan

REG NO: 192521421

COURSE CODE: CSA0918

Q) An inventory team creates `List<Object>` expecting it to be a substitute for `List<String>`. Explain why Java generics are invariant and redesign the API to accept lists of different element types safely. [Generic Invariance]

The challenge:

- In Java, `List<Object>` **cannot** be used as a substitute for `List<String>`, even though `String` is a subclass of `Object`.
- This is because Java generics are **invariant**. If Java allowed this, unsafe data could be inserted into the list.

Solution Strategy

- ❑ Create a `List<String>` containing string values.
- ❑ Do not assign it to `List<Object>` because generics are invariant.
- ❑ Use `List<?>` when the API only needs to read elements of different types.
- ❑ Use `List<? extends Object>` to safely accept lists of any object type.
- ❑ Display the elements using the redesigned method.
- ❑ Use `List<? super String>` when the API needs to add `String` values.

PROGRAM:

```
import java.util.*;
```

```
public class GenericInvariance {
```

```
    public static void displayList(List<?> list) {
```

```
        System.out.println("List elements:");
```

```
        for (Object item : list) {
```

```
            System.out.println(item);
```

```
}  
}
```

```
public static void addStrings(List<? super String> list) {  
    list.add("Laptop");  
    list.add("Keyboard");  
    list.add("Mouse");  
}
```

```
public static void main(String[] args) {
```

```
    List<String> inventory = new ArrayList<>();
```

```
    inventory.add("Monitor");
```

```
    inventory.add("Printer");
```

```
    inventory.add("Scanner");
```

```
    System.out.println("Original String List:");
```

```
    displayList(inventory);
```

```
    List<Integer> numbers = new ArrayList<>();
```

```
    numbers.add(10);
```

```
    numbers.add(20);
```

```
    numbers.add(30);
```

```
    System.out.println("\nInteger List:");
```

```
    displayList(numbers);
```

```
    List<Object> objectList = new ArrayList<>();
```

```
    addStrings(objectList);
```

```
        System.out.println("\nObject List after adding Strings:");
        displayList(objectList);
    }
}
```

OUTPUT:

Original String List:

List elements:

Monitor

Printer

Scanner

Integer List:

List elements:

10

20

30

Object List after adding Strings:

List elements:

Laptop

Keyboard

Mouse